ChatGPT

Roadmap: Building a Retrieval-Augmented QA System for Financial Filings

Stage 1: Document Ingestion (PDF & HTML SEC Filings)

Begin by **collecting and reading the financial filings** (10-K, 10-Q, etc.) from their source. The user has already downloaded a mix of PDFs and HTML files for companies like Apple, Meta, Tesla, etc. Key tasks in this stage:

- **PDF Parsing:** Use robust PDF text extraction libraries to ingest PDF filings. Common choices include pdfminer.six or PyPDF for text extraction [1]. These libraries can programmatically read PDF content into raw text. For more complex layouts (e.g. multi-column pages), consider `PyMuPDF` (MuPDF) or the **Unstructured** library, which can handle PDFs and other formats with minimal setup.
- **HTML Parsing:** For HTML filings, use an HTML parser like **BeautifulSoup** (from the `bs4` library) to load the HTML and extract the text content while stripping away HTML tags. BeautifulSoup can navigate the HTML DOM and pull out text from paragraphs, table cells, etc. If the HTML filings are from EDGAR, be mindful of irregular tags or nested tables [2]. In some cases, the **Unstructured** library's HTML partitioner can also be used for a consistent parsing approach across PDFs and HTML.
- **File Handling:** Implement a module or script that iterates over the downloaded files directory, detects file type (e.g. by extension or MIME type), and applies the appropriate parser. This ingestion step should output the raw text of each filing for further processing. Consider normalizing encoding (e.g., ensure UTF-8 text) and handling any file read errors or corrupt files gracefully.

Tools & Libraries: `pdfminer.six`, `PyPDF2/PyPDF`, `PyMuPDF (fitz)`, `unstructured`, `BeautifulSoup4` for HTML, Python's built-in `glob` or `os` libraries to iterate files.

Stage 2: Text Extraction and Cleaning (Including Tables)

Once raw text is extracted, **clean and normalize the text** to prepare it for chunking and embedding. Financial filings are complex and often include headers, footers, tables, and irregular spacing, so thorough preprocessing is crucial [3]. Key steps:

- **Remove Boilerplate and Artefacts:** Strip out page headers/footers, page numbers, line breaks hyphenating words, and any repetitive boilerplate text. For example, 10-K PDFs often have page numbers or section titles repeated on each page – remove these to avoid misleading the QA system [2]. Ensure that sentences broken by newlines (due to PDF layout) are concatenated properly into one line.
- **Standardize Whitespace and Encoding:** Normalize whitespace (e.g., replace multiple spaces or newlines with single spaces) and fix encoding issues (such as weird characters from PDF extraction). Convert all text to a consistent case (lowercase) only if it doesn't lose information (often not necessary for embeddings, as models handle case).

- **Table Handling:** Financial filings contain tables (financial statements, etc.) that need special handling. You have a few options: (a) **Convert tables to text** by extracting each row as a line, using a delimiter (like commas or tabs) between columns, so the information is preserved in a readable form for embeddings, or (b) use libraries like `camelot` or `tabula-py` to extract table data if you need structured outputs. For HTML filings, you can leverage pandas `read_html()` or BeautifulSoup to extract table text. The goal is to include table content (important figures) in the text, but be mindful that large tables may not be very useful to embed wholesale. In some cases, summarizing or selectively extracting key values from tables could be beneficial (optional enhancement).
- **Consistency and Quality Checks:** Because filings vary by company and format [4] [5], implement checks to ensure the cleaned text looks correct. For example, verify that important sections (like **Business Overview**, **Risk Factors**, **Financial Statements**) are present in the text. If the PDF was a scanned image (older filings), apply OCR (e.g., Tesseract) to get text, but then expect a noisier output that needs heavy cleaning (fix garbled characters, ensure numbers are correctly captured [6]). Fortunately, most recent filings from EDGAR are text-based PDFs or HTML, so OCR may not be needed.

By the end of this stage, you should have each document as a clean, continuous block of text that's ready for chunking. This step is critical – messy input (e.g. misordered text due to multi-column layout or leftover HTML tags) can degrade the quality of downstream embeddings and retrieval. Invest time in handling edge cases (different section names, varying table structures, etc.) to make the system robust [4].

Best Practices: Keep an eye on memory usage when processing large files (you may stream-read large PDFs instead of loading entirely in memory). Modularize your cleaning functions (one for removing headers, one for line breaks, one for tables) so they can be reused or toggled as needed. Consider writing tests on a few sample filings to ensure the cleaning behaves as expected (e.g., check that "Item 1. Business" section text remains intact after cleaning).

Stage 3: Document Chunking Strategy

With clean document text in hand, the next step is to **split each document into chunks** that are suitable for embedding and retrieval. RAG systems work by retrieving relevant chunks, so defining the chunk size and boundaries is important [7] [8]. Key considerations:

- **Chunk Size:** Determine an optimal chunk size that balances context and focus. Larger chunks (e.g. several paragraphs or ~1000 tokens) preserve more context from the document (useful for answering broader questions) but risk diluting the chunk's relevance if it covers multiple topics [8]. Smaller chunks (e.g. a few sentences or ~200-300 tokens) tend to be about one specific point, improving the chance that a relevant chunk directly answers a specific query [9]. For a first iteration, a chunk size of around **500 tokens** (approximately a few paragraphs of text) is a good middle ground, but be prepared to experiment. Many implementations use 300-500 words or tokens as a starting point. If using OpenAI embeddings, note token vs character: 500 tokens ~ 700-800 characters on average (depends on text).
- **Overlap for Context Continuity:** To avoid cutting important information at chunk boundaries, use overlapping chunks [10]. For example, if using 500-token chunks, you might overlap 50 tokens between consecutive chunks. This way, an answer that falls on a boundary can still be captured in at least one chunk. An overlapping strategy ensures that referential statements (e.g. "the Company" referring to a name mentioned in previous paragraph) or half-sentences at the end of a chunk carry

into the next chunk [10]. The overlap can be a fixed number of sentences or tokens – e.g., reuse the last sentence of a chunk as the start of the next. Libraries like LangChain provide **TextSplitter** utilities (e.g., `RecursiveCharacterTextSplitter`) where you can specify chunk size and overlap.
- **Chunk by Semantic Sections (Optional):** In addition to fixed-size chunks, consider leveraging the document's structure. For instance, you might split by major sections (10-K sections like "Business", "Risk Factors", "MD&A", etc.) and then further subdivide those if they are too large. This way, chunks don't mix content from completely different sections. Captide's blog suggests segmenting filings by logical sections which can help focus retrieval [11]. You can detect section titles (they often begin with "Item 1", "Item 1A", etc., for 10-Ks) and use them as boundaries. Ensure each chunk still doesn't exceed your target size (split within sections if needed).
- **Maintain Metadata:** While chunking, record metadata for each chunk – e.g., which document it came from, the section or page number, etc. For example, you might keep a structure like: `{"company": "Apple", "doc_type": "10-K", "year": 2023, "section": "Risk Factors", "chunk_id": 5}` associated with chunk text. This metadata will be useful later for filtering and for displaying sources.

Implement the chunking with a reliable method. You could manually split by paragraphs or newline markers and accumulate tokens until you hit the limit, but a safer approach is to use NLP tools: e.g., use **NLTK** or **spaCy** to split text into sentences, then group sentences until the token count threshold is met. This ensures you don't split in the middle of a sentence. Many frameworks (LangChain, LlamaIndex) have utilities for this as well. By the end of this stage, each document is represented as a list of chunks (text segments), each with associated metadata.

Stage 4: Embedding the Text Chunks

With chunks ready, generate **vector embeddings** for each chunk using a state-of-the-art embedding model. These embeddings will encode the semantic meaning of the text in a high-dimensional vector space [12]. Consider the following when choosing an embedding solution:

- **Model Choice:** You have a few good options:
- *OpenAI embeddings:* For example, OpenAI's `text-embedding-ada-002` model is a popular choice that produces 1536-dimensional vectors [13] and is state-of-the-art in quality. Using OpenAI's API, you can embed each chunk by calling the embedding endpoint (with batching to speed up if needed). This option generally yields excellent semantic representations of text, at the cost of API usage fees and requiring internet access.
- *Cohere embeddings:* Cohere provides embedding models via API as well (e.g., `embed-english-v2.0`), which produce high-dimensional vectors (4096 dimensions for some models) with strong performance on semantic search. This is an alternative if you have a Cohere API key.
- *Hugging Face / Local models:* There are many open-source models (typically from the **SentenceTransformers** library) you can run locally. For instance, `all-MiniLM-L6-v2` (384 dims) or `multi-qa-MPNet-base-dot-v1` (768 dims) are well-known general-purpose models. Newer models like **E5-large** (from IntFloat) or **InstructorXL** offer very strong results, and domain-specific models (e.g., FinBERT or FinSentenceEmbeddings trained on financial text) could capture nuances of finance filings. Using Hugging Face Transformers with a GPU can embed chunks without external API calls – useful if data privacy is a concern or to avoid costs, albeit possibly slower than OpenAI's optimized API.

- **Embedding Procedure:** Once you pick a model, embed every chunk's text into a vector. If using an API (OpenAI or Cohere), you might send chunks in batches (keeping within token limits) and get back an array of vectors. If using a local model via HuggingFace, load the model (possibly a `SentenceTransformer` pipeline) and call it on each chunk or batch of chunks. This will produce, say, a Python list or numpy array for each chunk's embedding.
- **Ensure Consistency: Use the same embedding model for all chunks and for all future queries** [12]. This is critical: the user's question embedding must be in the same vector space as the document chunks. For example, if you embed documents with OpenAI's ada-002 model, you **must** also use ada-002 to embed the query – mixing embedding models would make similarity search meaningless [12]. Document this choice clearly in your code/config.
- **Normalize or Not:** Many embedding models (like ada-002 or SentenceTransformers) produce vectors that are already suitable for cosine similarity. You may choose to normalize vectors (unit length) if you plan to use dot-product for similarity, but if using cosine distance directly (as in pgvector's functions), you can use raw embeddings. The OpenAI embeddings, for instance, can be used as-is with cosine similarity.

As you embed chunks, store the resulting vector along with an identifier for the chunk (and perhaps the text itself temporarily) for loading into the database in the next stage. If you have a very large number of chunks, consider the time and cost: OpenAI's embeddings cost ~$0.1 per 1K tokens, so embedding thousands of pages is a few dollars. If local, ensure you have the compute to process them in a reasonable time (multi-threading or batching can help utilize a GPU).

Best Practice: Verify embedding quality by manually checking a few examples – e.g., take a chunk and a related question, compute the embedding for both and see if their cosine similarity is higher than for an unrelated question. Also, confirm the embedding vector dimensions match expectations (e.g., 1536 for ada-002) to avoid dimension mismatches when storing in the database [14].

Stage 5: Setting Up PostgreSQL + pgvector for Vector Storage

Set up a **PostgreSQL database with the pgvector extension** to serve as your vector store. The pgvector extension allows Postgres to store high-dimensional vectors and perform efficient similarity searches on them [15] [16]. Outline:

- **Database Installation:** Install PostgreSQL (if not already) and add the **pgvector** extension. For example, on a local setup you might run `CREATE EXTENSION vector;` in psql after installation [17]. Ensure you have a recent version of Postgres that supports pgvector (Postgres 14+ is recommended). You can use a Docker image (e.g., `ankane/pgvector`) or hosted solutions (Supabase, Neon) that come with pgvector enabled.
- **Table Schema:** Design a table to hold your chunk vectors and metadata. A typical schema uses: an `id` primary key, a `vector` column of type `vector` with a fixed dimension, a `text` column for the chunk content (you can store the full chunk text or a shortened version for reference), and a `metadata` column (JSON or separate fields) for info like company, year, filing type, etc [18] [19]. For example:

```
CREATE TABLE filings_chunks (
    id SERIAL PRIMARY KEY,
    company TEXT,
```

```
    filing_type TEXT,
    year INT,
    chunk_id INT,
    text TEXT,
    embedding vector(1536)  -- assuming 1536-dim embeddings
);
```

Alternatively, use a JSONB `metadata` column to store a dictionary of metadata per chunk [19]. Make sure the vector dimension matches your model (e.g., 1536 for ada-002, or 768/1024/etc. as needed) [14]. If you prefer an ORM, **SQLAlchemy** has support for pgvector (via `pgvector.sqlalchemy.Vector` type) [20] [21], which can make defining the model and schema easier in Python.

- **Inserting Data:** Once the table is created, insert each chunk's embedding and data. Using Python, you can do this with `psycopg2` or SQLAlchemy. For example, with SQLAlchemy you'd create a `Vector` object for each chunk and add it to a session [22] [23]. If using psycopg2, you can use parameterized INSERT statements. It might look like:

```
insert_sql = "INSERT INTO filings_chunks (company, filing_type, year, chunk_id,
text, embedding) VALUES (%s, %s, %s, %s, %s, %s)"
cur.execute(insert_sql, (company, type, year, chunk_idx, chunk_text,
embedding_vector))
```

Note: You might need to adapt how you pass the vector (pgvector allows vectors to be passed as lists in JSON or using `pgvector` Python library). The **pgvector Python library** can help with binding vector parameters, or you can convert the numpy array to a Python list and use a specialized adapter.

- **Indexing (Optional for Large Data):** By default, pgvector performs exact nearest-neighbor search, which scans through vectors to find the nearest – this guarantees accurate results but can slow down if you have millions of vectors [24]. If your dataset is large and query latency becomes an issue, consider adding an approximate index. pgvector supports **IVFFlat** and **HNSW** indexes for faster search [24] [25]. For example, you could run:

```
CREATE INDEX ON filings_chunks USING ivfflat (embedding vector_ops) WITH (lists
= 100);
```

This creates an IVF index with 100 "lists" (tune this parameter as needed). HNSW indexing is also supported (as of pgvector 0.4+) and can be created similarly. Keep in mind approximate indexing introduces a trade-off: slightly less accuracy (it might miss a true nearest neighbor occasionally) in exchange for much faster query times [24]. For a personal project with moderate data, you may not need an index right away, but it's good to know for scaling.

At the end of this stage, your Postgres database acts as a **vector database**, containing all chunk embeddings and their associated info. This integrates with the rest of your stack seamlessly – you can use SQL queries to retrieve relevant chunks by similarity, as described next.

Suggested Stack: PostgreSQL 15+, pgvector extension; SQLAlchemy (for model/ORM, optional); psycopg2 or asyncpg for executing queries. The advantage of pgvector is you keep everything in one place (structured data and vectors) [26], and you can use SQL to join or filter by metadata easily, which we'll leverage for metadata filtering.

Stage 6: Semantic Vector Retrieval

Implement the **semantic retrieval pipeline** to fetch relevant chunks in response to a user's natural language query. This is the core of RAG: given a query, find the most semantically similar pieces of text from the filings. Steps in this stage:

- **User Query Embedding:** When a user asks a question, convert that query into an embedding vector using the **same embedding model** used for the documents [12]. For instance, if a user asks *"What were Apple's main risk factors in 2022?"*, call the embedding model (OpenAI, Cohere, or local) to get the query's vector representation. This yields a vector in the same 1536-dim space (if using ada-002, for example). This step is usually very fast (single short query to embed).
- **Similarity Search in PG:** Query the Postgres vector store for the top *k* most similar chunks to the query vector. In SQL, using pgvector, you can do:

```
SELECT company, filing_type, year, chunk_id, text,
       1 - (embedding <=> query_embedding) AS similarity
FROM filings_chunks
ORDER BY embedding <-> query_embedding
LIMIT 5;
```

Here `<->` is the operator for Euclidean distance (L2) by default; pgvector also provides `<=>` for cosine distance if vectors are normalized. You can choose to normalize all vectors and use cosine, or use L2 as is – in practice both can work (cosine is often preferred for text embeddings). The query above returns the top 5 chunks, and we compute a similarity score (larger is more similar) as `1 - distance` [27]. In Python with SQLAlchemy, as shown in the CodeAwake example, you might use `Vector.vector.cosine_distance(query_vec)` in the query and order by that [27]. This retrieves the most relevant chunks from the knowledge base for the query.

- **Metadata Filtering (Optional at Query Time):** If the user or UI specifies filters (e.g., only show results from a certain company or document type), incorporate that in the SQL `WHERE` clause. For example, to limit to Apple's filings: `... FROM filings_chunks WHERE company = 'Apple' ORDER BY embedding <-> query_vec LIMIT k;`. If using JSON metadata, you can query `WHERE metadata->>'company' = 'Apple'`. Pgvector allows seamless combination of vector search with relational filters [26] [28], which is a big advantage of using Postgres. We will discuss metadata filtering more in Stage 9, but ensure your retrieval function can accept optional filters.

- **Chunk Post-Processing:** Take the top *k* chunks (e.g., k=3 or 5) returned. You might want to do minor post-processing like sorting them by similarity (though the SQL already did) or even **re-ranking** them. A possible enhancement is to use a cross-encoder reranker: feed each retrieved chunk with the query into a smaller BERT-based model to get a relevance score, and sort by that. This can improve precision but at the cost of extra computation (we'll mention in enhancements). For the initial system, the vector similarity ordering is usually sufficient.

- **Return Chunks for Context:** The retrieval function should output not just the chunk text, but also any metadata or identifiers (e.g., which filing it came from, similarity score if you want to display it, etc.). This will be used by the LLM in the next stage, and potentially shown to the user as source references.

At this point, given a user query, you can obtain the top relevant chunks of filings that likely contain the answer or information related to the query. This semantic search allows the system to **narrow down the knowledge base** to just a few pertinent excerpts, which is far more efficient than feeding entire documents into an LLM. Modern vector search is very fast – with indexes or moderate data, these queries return in tens of milliseconds, even across millions of vectors [24].

Note: It's often useful to tune the number of chunks *k*. If k is too low, you might miss some relevant info (higher recall by increasing k). If k is too high, you'll feed a lot of possibly irrelevant text to the LLM (lower precision). A typical range is 3-5 chunks for a single query, which usually fits well within an LLM prompt.

Stage 7: LLM Response Generation (RAG QA over Retrieved Context)

Now comes the **Generation** part of Retrieval-Augmented Generation. Use a Large Language Model to formulate an answer to the user's question, *augmented* with the retrieved context. Essentially, we'll prompt an LLM with the top chunks from Stage 6 plus the question, so that it can "ground" its answer in those chunks [29]. Steps to implement:

- **Prompt Construction:** Construct a prompt that provides instructions, the retrieved context, and the user's question. A good prompt will explicitly tell the LLM to use the given context and not to fabricate information [30] [31]. For example:

```
System: You are a financial analysis assistant. Use the provided context to
answer the question. If the answer is not in the context, say you don't have
that information.
User:
QUESTION: <user's question here>
CONTEXT:
<chunk 1 text>

<chunk 2 text>

<chunk 3 text>

(…)
```

Then instruct the model to produce an answer. In OpenAI API terms, you'd have a system message with instructions, then a user message containing the question and context formatted as above. Rohit Aggarwal's RAG example prompt illustrates this format: *"Give an answer for the question using only the given context. If the information needed isn't in the context, acknowledge this rather than guessing."* [30]. The key is to **include all top retrieved chunks** (possibly separated by delimiters like `---` or newlines) and make it clear to the model that these are the reference materials. This helps ground the model's generation in actual data [31], greatly reducing hallucinations.

- **Choosing the LLM:** For a high-quality project, **GPT-4** via OpenAI API is an excellent choice (it will yield detailed, accurate answers when given context). GPT-3.5-turbo can also be used for cost efficiency, but GPT-4 generally is better at understanding and using the provided documents. If you prefer an open-source model (for local running or privacy), consider a fine-tuned Llama 2 (e.g., Llama-2 70B Chat, if you have the compute) or smaller ones like Llama-2 13B Chat for simpler Q&A. Other instruct models like **Anthropic Claude**, **Google PaLM 2 (via API)**, or open models like **GPT4All** or **Falcon-Instruct** could be used depending on resources. Make sure the model's context window is sufficient for the size of your chunks * k + the question. For example, GPT-4 (8k context) is usually fine for a few chunks; if you expect very long context, GPT-4 32k or Claude 100k might be considered, but typically not needed for filings Q&A.
- **Generating the Answer:** Send the prompt to the LLM and get the answer. If using OpenAI API, you'd call `openai.ChatCompletion.create(...)` with the messages. If using a local model via HuggingFace, you'd use the model's generate function with the combined prompt text. The LLM will produce an answer that ideally references the provided context. It might even quote or implicitly use phrases from the chunks. Design the prompt to encourage the model to produce a **detailed and well-structured answer** (since the user wants a high-quality result), but also to refrain from irrelevant info. As a safeguard, instruct it to say it doesn't know if something isn't in context.
- **Streaming vs. Full Completion:** Decide if you want to stream the answer to the UI (some front-ends like Streamlit or custom websockets can handle token streaming for a responsive feel) or just wait for the full completion. For the first version, getting the full completion and then displaying is simpler.
- **Multi-turn Interaction (optional):** Initially, treat each question independently. But if you plan a chat interface with follow-up questions, you might need to carry conversation context. This complicates retrieval (you may need to combine new query with previous answer or context). As an advanced feature, you could allow follow-ups by reusing the last chunks or refining them, but for now, assume one query -> answer at a time (which fits the typical QA setting).

After this stage, the user receives an **answer** that the LLM formulated, **grounded by actual data** from the filings. The quality of the answer will depend on the relevance of retrieved chunks and the LLM's capabilities. If the answer seems off or hallucinated, you may need to refine the prompt or retrieval (e.g., make instructions stricter or retrieve more specific chunks). The RAG approach ensures that if the info *is* in the documents, the model can use the phrasing and details from those documents to answer. For example, it might cite specific figures or use language from a risk factor description directly.

Best Practices: Monitor the LLM's output for any tendency to drift beyond the context. In internal testing, deliberately ask a question that is not answerable from the docs (e.g., "What is Tesla's CEO's favorite food?" if not in filings) and ensure the system responds with a disclaimer ("the information is not in the documents") rather than an invented answer. This will validate your prompt's effectiveness in avoiding hallucination [31]. You can also insert subtle known errors in context to see if the model correctly relies on context or not (as a way to gauge faithfulness).

Stage 8: Front-End Application (UI for Upload, Query & Results)

Create a **user interface** that ties everything together in an intuitive way. The UI should allow document uploads (to add new data) and querying (asking questions) with clear presentation of answers and sources. You have two broad approaches to building the front-end:

- **Web App with React (and Backend API):** A React (or Next.js) front-end can offer a dynamic, responsive UI. You would build components for: document upload (a form to upload PDF/HTML files,

which calls a backend endpoint to process and insert them into the database), a query input (text box for questions), and display of answers with sources. The React app would communicate with a backend (which you can implement in Python using **FastAPI** or **Flask**). For instance, an upload could `POST /ingest` with the file, which your backend processes through Stages 1-5 pipeline and stores in PG. A query would `POST /query` with the user question and optional filters, the backend runs Stage 6-7 (retrieval + LLM), and returns the answer and retrieved chunks. The front-end then renders the answer, and perhaps underneath, lists the sources (e.g., "Source: Apple 2022 10-K, Risk Factors section") with maybe an expandable snippet or highlight of the chunk text. This approach is more involved (two components to develop), but as an advanced developer, you get full control over the UI/UX. You can make it "impressive" by adding features like syntax highlighting for sources, interactive filters, or saving query history.

- **Streamlit App (All-in-One):** Streamlit allows you to write a Python app that directly creates a web interface. It's very convenient for prototypes and personal projects. You can use Streamlit file uploader (`st.file_uploader`) to handle uploads, and on upload, run the ingestion pipeline to process the file and update the vector DB. For Q&A, you can have a text input (`st.text_input`) for questions and a button to submit, then call your retrieval + LLM answer function, and finally display the answer. Streamlit can display the retrieved chunks perhaps in an expandable section or as Markdown text. While Streamlit might not look as polished as a custom React app, it can still be made modern with a bit of HTML/CSS tweaks and is much faster to develop since it's just Python. It's a good choice for a **personal project demo** – you can get a functional UI with minimal effort and focus on the ML pipeline.

UI Features to Implement:

- *Document Upload:* As mentioned, the UI should allow adding new documents. When a user uploads, say, a PDF, the system should run preprocessing, chunking, embedding, and insert those into the database (Stages 1-5). If using Streamlit, this can happen in the app script (though be mindful of long processing times – you might want to acknowledge file upload and maybe perform heavy processing in a background thread or show a progress bar). In a React setup, the backend would do this and respond when done.
- *Query Interface:* A simple input box for questions. You might also present example questions to guide the user. On submit, trigger the retrieval and LLM steps. The UI should handle the case where no answer is found (LLM says it doesn't have info) – maybe show a message that the info isn't in the docs.
- *Display of Results:* Show the answer from the LLM prominently. Below or alongside it, display the **retrieved context chunks** that were used. This is important for transparency: the user can see exactly which parts of the filings the answer is based on. You might label each chunk with its source (company, year, section). For example: "**Source 1 (Tesla 2023 10-K, Item 1A)**: [snippet of text]". If the chunk text is long, show maybe just the most relevant snippet or highlight the part that answers the question (you can do a simple keyword highlight of query terms in the chunk text for clarity). If feasible, link out to the original document (if it's publicly accessible or maybe to a local viewer).
- *Metadata Filter UI:* If implementing Stage 9 (filters), include dropdowns or multi-select for things like Company or Filing Type so the user can narrow the search context. For instance, a dropdown populated with company names present in the database, or a checkbox list for "10-K", "10-Q". If the user selects these, your query backend can use that to filter results as described earlier. This makes the app more interactive and powerful, especially if you have many documents.
- *Design and Responsiveness:* Aim for a clean, professional look. If using React, consider a component library (like Material-UI or Ant Design) for styled components. If using Streamlit, you can leverage the

built-in themes or custom CSS for a polished appearance. Possibly include your own branding or a title like "Financial Filings Q&A".

The front-end is what will make your project **impressive** to users: demonstrate that it's not just an offline script, but an interactive tool. Emphasize usability (clear instructions, error handling for bad inputs, etc.). For example, if a user asks a question unrelated to any document, handle it gracefully (LLM says it's not in context, and you can show "No relevant information found in the uploaded filings."). Logging user queries and responses (for your own analysis) can be helpful, but since this is personal, it's optional.

Stage 9: Metadata-Aware Filtering and Querying (Optional Enhancement)

Incorporating **metadata filtering** improves user control and result relevance, especially as your collection grows. By metadata, we mean attributes like company name, document type (10-K vs 10-Q), year or quarter, etc., which you associated with each chunk. How to support it:

- **Storing Metadata:** As done in Stage 5, ensure each chunk entry has metadata stored (either in separate columns or a JSONB column). For example, a chunk from *Apple's 2021 10-K* might have `company="Apple", year=2021, filing_type="10-K"`. If you've stored this in a JSON column called `metadata`, the structure might be `{"company": "Apple", "year": 2021, "type": "10-K", "section": "MD&A"}` for that chunk [19].
- **Backend Filtering Logic:** Update your retrieval query builder to include filters if specified. For instance, if the user selects company=Apple and type=10-K, your SQL query would add `WHERE company = 'Apple' AND filing_type = '10-K'` before the ORDER BY. If using SQLAlchemy, you'd add filter conditions like `.filter(Vector.company == "Apple", Vector.filing_type == "10-K")` before executing [26]. Because the vector similarity search is integrated, Postgres will first apply the filter to narrow down vectors, then compute distances, which is efficient (especially if you also index those columns, though not mandatory for moderate data).
- **UI Controls:** As mentioned in Stage 8, provide UI elements for users to select filters. This could be dropdown menus for company and filing type. Populate them dynamically (e.g., query your DB for distinct company names to fill the options). If using Streamlit, you might use `st.multiselect` for companies or similar widgets.
- **Use Cases for Filtering:** A user might want to restrict queries to a specific company's filings or compare answers between companies by switching filters. Or perhaps limit to annual reports only. This feature adds a layer of **data slicing** capability to your QA system. It's optional but highly useful as your knowledge base spans multiple organizations and document types. It also helps avoid confusion (e.g., if two companies have similar contexts, a question like "What are the main risk factors?" would retrieve chunks from all companies unless filtered – which might be fine, but if the user intended only one company, a filter clarifies it).

Implementing metadata filtering will make your system feel more complete and professional. It demonstrates foresight that as the corpus grows, users can zero in on subsets of data. In the context of pgvector, this is straightforward and one of its advantages over some pure vector DBs that might need separate metadata filtering steps.

Stage 10: Evaluating Retrieval Quality and System Performance

For a high-quality project, it's important to **evaluate how well the system is working**, especially the retrieval component. The retrieval quality directly affects the accuracy of answers (garbage in, garbage out). Set up a simple evaluation regimen:

- **Ground-Truth Q&A Pairs:** Create a small test dataset of questions and answers based on the documents. Since this is a personal project, you can curate this manually. For example, take each filing and formulate a few questions that you know the answer to (from that text). Write down the question and the exact answer or the chunk that contains it. This serves as ground truth to test your system. Ensure the questions vary (some factual, some that require synthesis of a couple of chunks).
- **Retrieval Metrics:** Use **Precision@K** and **Recall@K** to evaluate the retriever [32]. For each question in your test set, run the retrieval (Stage 6) to get, say, top 5 chunks. See how many of those chunks are actually relevant to the question (i.e., contain the answer). *Precision@5* would be (# of relevant chunks in top 5) / 5. *Recall@5* would be (# of relevant chunks in top 5) / (total # of relevant chunks in the whole corpus for that question). If you crafted one chunk as the known answer, recall is basically whether that chunk was retrieved or not. Average these metrics over your test questions. This gives a sense of how well the vector search is doing in finding the right info [33] [34]. For instance, a high recall means the system usually finds the answer somewhere in the top K chunks (good, because then the LLM has what it needs). A high precision means most of the top chunks are actually useful (also good, as it means less noise fed to the LLM).
- **Evaluate End-to-End QA:** Beyond retrieval, you might also evaluate the final answers qualitatively: Are they correct and based on the documents? One way is to check if the answer text contains statements that can be directly traced to the source chunks. If you want to be rigorous, you could use metrics like **BLEU** or **ROUGE** between the LLM's answer and a reference answer, but in QA these can be misleading. Manual evaluation or using another LLM to judge correctness can be done. Since the project is personal, you can manually review answers for a set of test questions. Ensure that when the answer is wrong or not findable, the system says it doesn't know (which is a form of correctness – avoiding a confident wrong answer).
- **System Performance:** Evaluate speed – how long does it take from question to answer. Vector search in Postgres is typically fast (a few hundred milliseconds or less for moderate data). The bigger latency is the LLM response. GPT-4 might take a few seconds to produce an answer, especially if lengthy. That's usually fine for a user-facing app (a 5-second response is acceptable for complex queries). If you need faster answers, GPT-3.5 or local models might be faster but possibly at cost of quality. Monitor memory and CPU usage as well, especially if using local models. If embedding on the fly or processing uploads, ensure that doesn't block other operations excessively (you might queue document ingestion tasks).

Use the evaluation results to iterate: for example, if you find recall@5 is low, you might increase chunk size or overlap, or try a different embedding model that better captures the domain, or increase K. If precision is low (many irrelevant chunks in top results), you might try filtering obvious off-topic chunks (perhaps by requiring some keyword overlap between query and chunk as a secondary heuristic – a simple trick for precision). This kind of tuning can significantly improve the end QA quality.

Additionally, consider user feedback as an evaluation source: if a user (even if it's just you or colleagues testing) asks a question and the answer is wrong, investigate why (was it retrieval miss or LLM error?). This can guide adjustments. Keep a log of queries and what was retrieved/answered (if possible) to analyze later.

Metrics Summary: Aim for high *Recall@K* for the retriever (to ensure relevant info is found) and acceptable *Precision@K* (so the LLM isn't seeing too much junk). Mean Reciprocal Rank (MRR) is another metric: it measures how far down the rank the first relevant result is on average [32]. A higher MRR (closer to 1.0) means the answer chunk is usually ranked very high. These metrics will give an academic credibility to your project and help demonstrate its effectiveness.

Stage 11: Optional Enhancements and Best Practices

Finally, consider some **enhancements and best practices** to take your RAG system from good to great. These are optional, but implementing them can make your project more comprehensive and impressive:

- **Hybrid Search (Lexical + Semantic):** Improve the retrieval by combining vector search with traditional keyword search. Sometimes a user query has specific keywords (e.g., a particular term or figure) that a pure vector search might not prioritize if the embedding doesn't heavily weight that exact token. Implementing a hybrid search can catch such cases. For example, use Postgres's **full-text search** (tsvector/tsquery or a simpler `ILIKE '%keyword%'`) alongside vector search and intersect results or use a scoring fusion. A simple approach: filter vector results by those that contain at least one query keyword to boost precision. A more advanced approach: retrieve top N by vector, top M by keyword (BM25 via an extension or just basic search), then merge and re-rank. Hybrid search gives the "best of both" – semantic understanding plus exact keyword matching [35] [36]. This can especially help for things like specific numerical values or names that vector models might not distinguish well. If interested, libraries like Elastic or Whoosh could be used, or since you are in Postgres, you can use the built-in full-text search or the **ParadeDB** extension which brings BM25 to Postgres [37].
- **Summary Generation:** You might add a feature to **summarize a document or answer** broader questions. For instance, "Give me a summary of Apple's 2022 10-K." You can implement this by feeding the entire document (in chunks) through an LLM in pieces and asking it to summarize, or more simply, after retrieving many chunks for a broad query, have the LLM produce a consolidated summary. Another angle is to pre-generate summaries for each filing upon ingestion (store them as metadata), so that the system can output a ready summary on demand or use it to answer high-level questions. Summarization can be done with the same LLM (maybe using a prompt like "Summarize the following text…") or using specialized models (like GPT-3.5 or Pegasus for summarization). This adds value for users who might want a quick overview instead of a detailed answer.
- **Chunk Reduction or Refinement:** If your chunks are long or numerous, you can implement a second-stage retrieval or refinement. For example, retrieve 10 chunks, then have a smaller model or heuristic filter them down to the most relevant 3 (perhaps by checking which chunks have the most overlap with the query terms, or by asking an LLM which chunks seem most likely to contain the answer). This two-step approach (recall then precision) can help in cases where the initial embedding recall is good but precision is low. Reranking using a cross-encoder (like Microsoft's MPNet cross-encoder or MiniLM cross-encoder trained on MS MARCO) is a known technique: it treats the query and chunk text as a pair and scores relevance, often improving the ordering of top results. This could boost answer accuracy [38].
- **Use Domain-Specific Models:** Consider using or fine-tuning models on financial text for better performance. For example, there are financial phrase embedding models (some use FinBERT or other finetuned Transformers on SEC filings). If you find the general embedding isn't capturing some nuances (maybe terms like "EPS" or "Form 10b5-1" etc.), a domain model might place related concepts closer in vector space. Similarly, for the LLM, OpenAI's GPT-4 is general but very strong;

however, Bloomberg has a **BloombergGPT** (not publicly available) or you could fine-tune a Llama on financial QA if you had the data. This might be overkill for a personal project, but worth mentioning as a direction.

- **Monitoring and Logging:** Implement logging for queries, retrieval results, and responses. This will not only help with evaluation as discussed, but also is a best practice for any QA system. It allows you to audit what sources were used for an answer (important for trust) and debug issues. You could even build an admin dashboard (maybe using Streamlit or a Jupyter notebook) to inspect the vector DB content or search results for a given query.
- **User Feedback Loop:** (If this were a live project) let users mark if an answer was helpful or not, and use that to improve. For example, if a user says an answer was wrong, you could trace if it was due to a bad chunk retrieved or the model. Over time this could guide improvements (like maybe exclude certain boilerplate chunks that often confuse the model, etc.).
- **Security and Privacy:** Since this is a personal project on public data, there aren't major privacy concerns. But as a best practice, remember that if using closed APIs (OpenAI/Cohere), your data (filings text and queries) is leaving your environment. Ensure that's acceptable (OpenAI for instance does not train on your data by default and has policies in place). If this were for internal/company data, you'd likely choose a self-hosted model and vector store for security [39]. Mentioning this consideration shows awareness of deployment contexts, which is good in documentation.
- **Scalability Considerations:** If you were to scale this system up (say dozens of companies, years of filings), think about how you'd handle thousands of documents. Postgres can handle millions of rows, but you'd want to ensure indexing is in place and perhaps partition the data by company or year (to speed up filtered searches). Also, embedding all filings can be time-consuming – you might implement a more incremental approach (embed on the fly when a new doc is uploaded, which we have, but also maybe pre-embed known sets in batch offline). Using a message queue or background worker for ingestion is a pattern that can keep the UI responsive. These are not necessary for the prototype but are good practice as project grows.
- **Documentation & Readme:** As part of a professional project, document the setup clearly. Include instructions on how to install pgvector, how to run the app, how to get API keys for OpenAI or others, etc. Provide examples of queries in your README. This will make your project stand out as complete and developer-friendly.

By implementing the above enhancements and following best practices, your RAG system will not only answer questions accurately but will be robust, transparent, and maintainable. Retrieval-Augmented Generation is a cutting-edge approach in NLP, and applying it to financial filings is both challenging and rewarding – with this roadmap, you're set to build a powerful personal QA assistant for financial documents, showcasing skills in data engineering, NLP, and full-stack development. Good luck with your project, and enjoy the deep insights you'll gain from those 10-Ks and 10-Qs along the way!

Sources:

1. Rohit Aggarwal, *"Exercise 1: Simple RAG for 10-K filings,"* *Medium*, Mar. 2025 – Explanation of RAG fundamentals (chunking, embeddings, vector DB) with 10-K examples [8] [10].
2. *CodeAwake Blog – "PostgreSQL with pgvector as a Vector Database for RAG,"* Oct. 2024 – Detailed tutorial on using pgvector for RAG, including setup, chunk insertion, and similarity search in SQL [40] [24].
3. *LangChain Blog – "Kay, Cybersyn, LangChain: Embedding SEC Filings for RAG,"* Oct. 2023 – Discusses challenges of processing financial filings (tables, chunking, accuracy needs) [3].
4. Captide (Fintech) Blog, *"How to Extract Data from Financial Filings,"* 2023 – Overview of pipeline for parsing filings, noting cleaning difficulties (formats, OCR, inconsistency) [2] [5].

5. Adrian Araya, *"How to Evaluate RAG Systems," Ridgerun*, Nov. 2024 – Guide on RAG evaluation metrics like Precision@K, Recall@K, MRR for retrieval quality [32] [34].
6. *PostgreSQL + pgvector Documentation and Examples* – pgvector GitHub and Datacamp tutorial (2023) showing how to create vector columns and perform similarity queries in SQL [22] [27].

---

[1] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [40] PostgreSQL with pgvector as a Vector Database for RAG
https://codeawake.com/blog/postgresql-vector-database

[2] [4] [5] [6] [11] Captide | How to Extract Data from Financial Filings
https://www.captide.ai/insights/how-to-extract-data-from-financial-filings

[3] Kay x Cybersyn x LangChain: Embedding SEC Filings for RAG
https://blog.langchain.com/kay-x-cybersyn-x-langchain/

[7] [8] [9] [10] [12] [29] [30] [31] [38] Exercise 1: Simple RAG for 10-K filings | by Rohit Aggarwal | MAdAiLab | Medium
https://medium.com/madailab/exercise-1-simple-rag-for-10-k-filings-f4a15f5386f0

[13] What is the vector size of text-embedding-ada-002? - Milvus
https://milvus.io/ai-quick-reference/what-is-the-vector-size-of-textembeddingada002

[32] [33] [34] How to Evaluate Retrieval Augmented Generation (RAG) Systems
https://www.ridgerun.ai/post/how-to-evaluate-retrieval-augmented-generation-rag-systems

[35] Hybrid Search - Azure AI Search | Microsoft Learn
https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview

[36] [37] Hybrid Search in PostgreSQL: The Missing Manual - ParadeDB
https://www.paradedb.com/blog/hybrid-search-in-postgresql-the-missing-manual

[39] SurrealDB | Beyond black boxes - building customisable and secure RAG systems for financial services
https://surrealdb.com/blog/beyond-black-boxes--building-customisable-and-secure-rag-systems-for-financial-services